

Donation Campaigns

phpBB Extension — Complete Documentation

Version 1.0.0-beta4 · 2026-10-04

Contents

Donation Campaigns

- What it does
- What it does not do
- Requirements
- Installation
- Permissions
- Configuration
- Campaign workflow
- Donation workflow
- Known limitations
- Licence

Administrator guide

- Before you start
- 1. Install and enable
- 2. Grant the permissions
- 3. Configure the currency
- 3a. ⚠ Before you merge or split a topic
- 4. Create a campaign
- 5. Record a payment
- 6. Decide whether to name the donor
- 7. Edit or delete a confirmed entry
- 8. Recalculate a total
- 9. Disable or archive a campaign
- 9b. The board-wide campaign list
- 9a. Where actions are logged
- 10. What happens when a topic or forum is deleted

Troubleshooting

Privacy and data handling

- What is stored
- What is not stored
- The donor name is the sensitive part
- Deletion and erasure
- Logging

Donation Campaigns — Release Notes

- 1.0.0-beta4
- Changed: templates in native Twig
- Fixed
- Upgrade from 1.0.0-beta3
- For translators

Developer notes

- Package
- Layers
- Money
- The campaign total
- Transaction boundaries

Confirmed-donation semantics

The action button

Deletion cascades

Shadow topics

phpBB integration points

The escaping contract

The BBCode description

Styles

Tests

Local integration environment

EPV and coding standards

Donation Campaigns

A phpBB 3.3 extension that attaches a fundraising campaign to a topic and shows its progress above the first post.

- [Administrator guide](#) — the day-to-day workflow
 - [Privacy and data handling](#) — what is stored, and what publishing a donor name means
 - [Developer notes](#) — architecture, contracts, tests
-

What it does

An administrator — or any user or group you authorise for that forum — creates a campaign, points it at an existing topic, and sets a target amount. The topic then shows a box above its first post with the target, the amount collected, a progress bar, and — optionally — the number of donations and the names of donors who agreed to be named.

As money arrives, an administrator or an authorised user records each payment from the topic. The public total is the sum of those records and nothing else.

Optionally, a board-wide page (`app.php/donationcampaigns` , linked from the quick links) lists every campaign a visitor may see, with its progress. It is off until an administrator switches it on, and it names no donors.

What it does not do

Read this part before installing. It is the difference between what this extension is and what people often assume a "donation extension" is.

- **It does not process payments.** No PayPal, no Stripe, no card handling, no webhooks, no callbacks. The extension never contacts a payment provider and makes no outbound requests of any kind.
- **It records confirmed receipts only.** A donation row means money that has *already arrived* and that an administrator has verified. It is bookkeeping, not a transaction.
- **No visitor or member can submit or confirm a donation.** There is no public form. Every entry is made from the topic by someone holding the appropriate permission — an administrator, or a user or group granted the forum-scoped donation permission for that forum.
- **It does not verify anything.** It cannot check a bank statement or a provider's records. Whether the money truly arrived is the administrator's judgement, and the extension records that judgement.
- **It stores no payment data** — no account numbers, no IBANs, no transaction identifiers, no card details, no proof-of-payment files. See [PRIVACY.md](#).
- **The action button is a plain external link.** Each campaign sets its own button text and destination — "How to donate", "Request bank details", "Über PayPal spenden". The destination may be a PayPal link, a page of bank instructions, another phpBB topic, a GoFundMe page or any other `https://` address. It is a link and nothing more: no provider script, no embedded form, no logo, no HTML. Whoever clicks it leaves the board, and the administrator still confirms the money by hand afterwards.

Using PayPal, or anything else

There is nothing to configure and no integration to enable. Create a hosted donation link in your own PayPal account, paste it into the campaign's **Donation link**, and set **Link text** to whatever the button should say — for example *Donate via PayPal*. The same field takes a GoFundMe or Betterplace page, a bank-information page, an internal phpBB topic, or any other valid `https://` address, and each campaign can point somewhere different.

The extension treats every one of these as an ordinary external link. It does not know which provider a URL belongs to, stores no merchant identifier, and performs no verification: whoever clicks the button leaves the board, and the administrator records the money by hand afterwards. That is the whole design, and it is intentional rather than unfinished — see [DEVELOPERS.md](#).

Requirements

phpBB	3.3.16 or later , below 4.0. Enforced by <code>ext.php</code> ; installing on an older 3.3.x fails cleanly
PHP	8.2 or newer
Database	See below
Style	prosilver
Language	English and German

Verified versus designed-for

Stating this precisely matters more than making the table look full.

	Actually executed	Designed for, not executed
phpBB	3.3.17	3.3.16 (the supported floor, but never run)
PHP	8.2 (board), 8.2/8.3 (tests)	8.4 — permitted by <code>>=8.2</code> , not run
Database	SQLite 3, MariaDB 10.11	MySQL, PostgreSQL, MS SQL Server, Oracle

The extension uses phpBB's DBAL throughout and writes no database-specific SQL, so the untested engines are expected to work — but "expected" is not "verified", and this table will not claim otherwise.

prosilver is the only officially supported style (ADR-013). Styles that inherit from prosilver may work; that is neither tested nor supported.

Installation

1. Copy the package to `ext/uflagmey/donationcampaigns/` in your board.
2. **ACP** → **Customise** → **Extensions** → **Donation Campaigns** → **Enable**.

Or from the command line:

```
php bin/phpbbcli.php extension:enable uflagmey/donationcampaigns
php bin/phpbbcli.php cache:purge
```

Lifecycle

Action	Effect
Enable	Creates two tables, seven configuration settings, the three permissions and their permission category, and the ACP menu
Disable	Hides the campaign box and the ACP menu. All data is kept , and re-enabling restores everything
Purge	Destroys all campaigns and donations , and removes the tables, settings and permissions. This cannot be undone
Upgrade	Replace the files and run <code>php bin/phpbbcli.php db:migrate</code> . Migrations are idempotent and safe to re-run

Take a database backup before purging. "Disable" is the reversible one; "purge" is not.

Permissions

Three permissions, grouped under a dedicated **Donation Campaigns** category in the permissions UI:

- **a_donationcampaigns** — administrator, **global**. Full access: the ACP settings and read-only oversight lists, the admin-only maintenance (recalculate a total; hard-delete a non-empty campaign), and — as an override — every frontend campaign and donation action on every forum. Granted to `ROLE_ADMIN_FULL` and `ROLE_ADMIN_STANDARD` on installation, if those roles exist.
- **f_donationcampaigns_manage** — forum permission, **forum-scoped**. Lets the holder manage the campaign *shell* on topics in that forum: create, edit, enable/disable, and delete an *empty* campaign. It does **not** grant donation management.
- **f_donationcampaigns_donations** — forum permission, **forum-scoped**. Lets the holder manage the donation *ledger* on topics in that forum: add, edit and delete confirmed donations. ⚠ This permission exposes donor names, private donor identities and confirmed amounts — grant it only to people you trust with that personal data.

The two forum permissions are **independent**: holding one does not grant the other. `a_donationcampaigns` overrides both; it is not granted on install to non-admins. **Read access is required for everyone**, including the administrator override: nobody manages a campaign in a forum they cannot read.

Any group can be authorised — moderators or not. The two permissions are ordinary *forum* permissions, assigned per forum on the **Forum permissions** tab (ACP → Permissions → Group's / User's forum permissions → Advanced permissions → Donation Campaigns), exactly like "Can post polls". Granting them gives the holder nothing else: no MCP access and no listing as a moderator. Neither is granted on installation.

Upgrading from 1.0.0-beta1: the beta1 moderator permissions (`m_donationcampaigns_*`) are replaced and their grants are **not** carried over. Re-assign the new forum permissions after updating. See [RELEASE NOTES BETA2.md](#).

Configuration

ACP → Extensions → Donation campaigns → Settings

Setting	Default	Notes
Currency code	<code>EUR</code>	Three letters, e.g. <code>EUR</code> , <code>USD</code> , <code>GBP</code>
Currency symbol	€	Up to 10 characters
Symbol before the amount	No	Yes: € 10,00 . No: 10,00 € . Applies to every amount shown
Space between symbol and amount	Yes	No gives \$10.00 . The space never wraps
Decimal places	2	0–4. 0 for yen, 3 for dinar
Donors listed	25	1–500, before the box summarises the rest
Show the campaign list page	No	Yes adds the board-wide list at <code>app.php/donationcampaigns</code> and a quick-links entry. It shows only campaigns the visitor could also see in their topic, and no donor names

⚠ **Changing "Decimal places" after donations exist changes how every stored amount is read, and converts nothing.**

An amount stored as `1000` shows as `10.00` at two places and `1.000` at three. The extension refuses the change unless you confirm it. See the [administrator guide](#).

Campaign workflow

Campaigns are managed **from the topic**, not the ACP.

1. Open the topic the campaign is for.

2. **Topic tools (the wrench)** → **Donation campaign**. This opens the management landing page for that topic; the topic is already fixed, with nothing to type and no ID to look up.
3. The landing resolves the current state: with no campaign yet (and permission to manage the shell) you get the create form; with a campaign already there you get its summary plus only the actions you are authorised for.
4. On the create/edit form, enter a title, an optional BBCode description, and the target; optionally add a donation link (`http://` or `https://` only); and choose whether to show donor names, the donation count and the date of each donation (the date is listed only when names are shown).
5. Save, then **Back to topic** — the box is there.

Or together with a new topic: when you start a topic in a forum where you may manage campaigns, the posting form has a **Donation campaign** tab next to the poll tab. Tick *Attach a donation campaign to this topic*, fill in the same fields (an empty title takes the topic title), and the campaign is created with the topic. Only for new topics; editing always goes through the topic tools. Drafts do not keep these fields.

Campaign actions (create, edit, enable/disable, delete an empty campaign) require `a_donationcampaigns` or `f_donationcampaigns_manage` in that forum. The ACP shows a read-only oversight list of campaigns and admin-only maintenance; it cannot create or edit one.

One campaign per topic, enforced by a unique database index. A campaign cannot be attached to the stub left behind by a moved topic — phpBB treats such a stub as non-existent, so a campaign there would be unreachable.

Donation workflow

1. **Wait for the money to actually arrive.**
2. On the topic, **Topic tools** → **Donation campaign** to open the management landing, then **Add confirmed donation**. The donation ledger — the button and the per-row Edit/Delete controls — is shown only to holders of `a_donationcampaigns` or `f_donationcampaigns_donations` in that forum; a manage-only holder does not see it.
3. Enter the amount received, the date it arrived, and the donor's display name.
4. Decide whether the donor may be named publicly — see [PRIVACY.md](#) before ticking that box.
5. Save. The campaign total is recalculated immediately from all its donations.

Amounts accept both `50.00` and `50,00`. Leave the name empty to record the donation as *Anonymous*. A donation marked private still counts towards the total and the count; only the name is withheld.

The total is always recomputed as the sum of the donation rows, never adjusted by arithmetic, so a figure that has somehow drifted is corrected by the next edit — or immediately, using **Recalculate total** in the ACP.

The ACP shows a read-only oversight list of donations; recording, editing and deleting them happens on the topic.

Known limitations

- No payment processing, and no provider-specific integration. Any provider can be linked to; none is integrated with. This is a design decision, not a gap — see [DEVELOPERS.md](#).
- No public donation form; every entry is made from the topic by an administrator or an authorised user.
- **Campaigns can only be managed from their topic.** A style that does not provide the `viewtopic_topic_tools_after` template event therefore offers no way to create one — see the prosilver note above.
- **A campaign cannot be moved to another topic** once created.
- Donation dates are date-only and stored as UTC midnight; the board's timezone is not applied to them.
- prosilver only.
- No bulk deletion of donations.
- The ACP campaign list, the donation list and the public campaign list paginate at 25 rows; there is no search and no filter.
- **Not yet reviewed in a browser across styles and screen sizes.**
- Databases other than SQLite and MariaDB have not been executed.

- The phpBB validation and development policies have not been reviewed, so no claim of compliance with them is made.

Licence

GPL-2.0-only. See [license.txt](#).

phpBB is a trademark of phpBB Limited. This extension is not affiliated with or endorsed by phpBB Limited.

Administrator guide

How to run a donation campaign with this extension, and what to be careful about. See the [README](#) for what the extension is, and [PRIVACY.md](#) for what publishing a donor's name means.

Before you start

This extension is a **ledger of payments you have already received**. It does not take money, does not talk to PayPal or a bank, and cannot tell whether a payment arrived. You check your account, and you record what you find.

That means one habit matters more than any setting here:

Record a donation only after the money has actually arrived and you have verified it yourself.

Everything the public sees — the total, the progress bar, the donor list — is built from those records. If you enter a promise, the board will show it as money received.

1. Install and enable

Copy the package to `ext/uflagmey/donationcampaigns/`, then **ACP** → **Customise** → **Extensions** → **Donation Campaigns** → **Enable**.

Enabling creates the tables, the settings, the three permissions and their permission category, and the ACP menu. Disabling later hides everything but keeps your data.

Updating to a newer version:

1. **Disable** the extension. Do *not* click "Delete data" — that removes every campaign and donation.
2. Upload the new files **over** the existing ones. Overwriting is safe; there is no need to delete the old folder first, and not deleting it means an interrupted upload leaves a working version behind.
3. **Check the upload is complete**. The extension folder must contain twelve folders: `acp`, `adm`, `config`, `controller`, `docs`, `event`, `exception`, `language`, `migrations`, `repository`, `service`, `styles`. FTP clients can skip folders without an obvious error.
4. **Enable** the extension — this runs any new database migrations — and purge the cache.

2. Grant the permissions

The extension ships three permissions, grouped under a dedicated **Donation Campaigns** category in the permissions UI.

`a_donationcampaigns` — administrator, granted globally. Full and Standard Administrator roles receive it automatically at installation. It governs the ACP (settings, the read-only oversight lists, and the admin-only maintenance: recalculate a total, hard-delete a non-empty campaign) and, as an override, every frontend campaign and donation action on every forum the administrator can read. An administrator can do everything from the topic as well as from the ACP.

The other two are **forum permissions, forum-scoped** — you grant them to any user or group, moderators or not, on the specific forums whose campaigns they should manage. They sit on the **Forum permissions** tab, next to phpBB's own "Can post polls":

ACP → **Permissions** → **Group's forum permissions** (or User's forum permissions) → choose the group and the forum(s) → **Advanced permissions** → category **Donation Campaigns**.

- `f_donationcampaigns_manage` lets the holder manage the campaign *shell* on topics in that forum: create, edit, enable/disable, and delete an *empty* campaign. It does **not** grant donation management.

- `f_donationcampaigns_donations` lets the holder manage the donation *ledger* on topics in that forum: add, edit and delete confirmed donations.

The two forum permissions are **independent**: holding one does not grant the other. `a_donationcampaigns` overrides both. Neither is granted on installation, and neither is part of any default role: assign them explicitly, or add them to a forum role of your own.

Read access is required. Whoever manages a campaign — including an administrator through the override — must be able to read the forum. Without *Can read forum* there, the management entry is not shown and every management page answers "not found".

Granting these permissions gives the holder nothing else: no access to the Moderator Control Panel, no other moderator ability, and no listing as a moderator of the forum.

⚠ `f_donationcampaigns_donations` **exposes personal data**. It reveals donor names, private donor identities and confirmed amounts. Grant it only to people you trust with that information.

Upgrading from 1.0.0-beta1. beta1 used two *moderator* permissions (`m_donationcampaigns_manage`, `m_donationcampaigns_donations`). The update removes them together with their grants and with the moderator standing they gave, and adds the two forum permissions above, granted to nobody. Re-assign them after updating. A custom permission role you built only for the old permissions is left in place but empty; delete it if you no longer need it.

3. Configure the currency

ACP → Extensions → Donation campaigns → Settings

- **Currency code** — three letters, e.g. `EUR`
- **Currency symbol** — shown next to every amount, e.g. `€`
- **Symbol before the amount** — *No* (default) gives `10,00 €`, *Yes* gives `€ 10,00`. It applies everywhere an amount appears: the topic box, the management pages, the ACP lists, confirmation dialogs and log entries.
- **Space between symbol and amount** — *Yes* (default) keeps them apart; *No* gives `$10.00`. The space is non-breaking, so an amount never wraps between number and symbol.
- **Decimal places** — `2` for most currencies, `0` for yen, `3` for dinar
- **Donors listed** — how many names the public box shows before summarising
- **Show the campaign list page** — see step 9b

⚠ Changing the decimal places later

This is the one setting that can silently misrepresent your figures.

Amounts are stored as whole numbers of the smallest unit — 250.00 € is stored as `25000` cents. The decimal-places setting decides where the separator is drawn when that number is displayed. **It converts nothing.**

Stored	At 2 places	At 3 places
<code>25000</code>	250.00	25.000
<code>870</code>	8.70	0.870

Change it on a board that already has donations and every historic amount is re-read, not re-scaled. The extension will not let you do this by accident: if any campaign or donation exists, it shows a warning and refuses the change until you tick a confirmation box.

Set this correctly **before** recording your first donation. If you must change it afterwards, expect to correct every stored amount by hand.

3a. ⚠ Before you merge or split a topic

Merging a topic into another, or splitting every post out of a topic, deletes that topic — and with it its campaign and every donation record you have entered. phpBB removes the emptied topic automatically, and this extension cleans up alongside it.

There is no warning and no undo, and a moderator can trigger it without holding the donation-campaign permission. If a topic carries a campaign, move posts *into* it rather than merging it away, or export the donation list first.

This is a known defect scheduled to be fixed before the final 1.0 release.

4. Create a campaign

Campaigns are created from the topic they belong to. There is no way to create one from the ACP, and no field anywhere that asks for a topic number.

Open the topic, then:

Topic tools (the wrench) → Donation campaign

That opens the campaign form with the topic already fixed. The same menu entry appears on every topic and always reads *Donation campaign*: if the topic has no campaign you get a new one to fill in, and if it already has one — even a disabled one — you get that campaign to edit. Which of the two you see is decided when you click, not when the page was drawn, so it is never out of date.

The entry shows to anyone who can manage the campaign shell **or** the donations in that forum — administrators (via the `a_donationcampaigns` override) and users or groups holding either `f_donationcampaigns_manage` or `f_donationcampaigns_donations` for that forum, provided they can read it. It is invisible to everyone else, including guests.

Field	Notes
Title	The heading of the public box
Topic	Shown, not editable. A campaign belongs to its topic for life
Description	Optional. BBCode, smilies and links are permitted
Target amount	<code>250.00</code> or <code>250,00</code> . Must be above zero
Donation link	Optional. Must be a full <code>http://</code> or <code>https://</code> address
Link text	The words on the public button, for example <i>How to donate</i> or <i>Über PayPal spenden</i> . Required whenever a donation link is set
Enabled	Untick to hide the box while keeping the data
Show donor names	See PRIVACY.md first
Show donation count	Shows how many donations, without naming anyone
Show donation date	Lists the day each donation arrived next to name and amount. Only with <i>Show donor names</i> . Ticked for new campaigns; campaigns that existed before 1.0.0-beta2 keep it off until you tick it

Save, and the page offers **Back to topic**, where the box now appears.

If someone else created or deleted a campaign for that topic while you had the form open, nothing you typed is saved. The form comes back with the current state and says what happened, so a colleague's campaign is never overwritten by a form that was drawn before theirs existed.

Create a campaign together with a new topic

When you start a **new topic** in a forum where you may manage campaigns, the posting form shows a **Donation campaign** tab next to *Poll creation*. Its panel has a checkbox, *Attach a donation campaign to this topic*, and the same fields as above (without the topic, which does not exist yet).

- Unticked, nothing happens, whatever the fields contain.
- Ticked, the fields are checked when you preview or submit, like a poll: a wrong target stops the post, the error appears at the top of the form and everything you typed is kept.
- Leave the title empty to use the topic title.
- The campaign is created right after the topic and logged like any other new campaign. If the topic needs approval, the campaign waits with it: nobody but moderators sees either until the topic is approved, and a disapproved topic takes its campaign with it.
- The tab is not offered when replying or editing — later changes go through *Topic tools* → *Donation campaign* as usual.
- **Drafts** keep only subject and message: the campaign fields are lost when you save a draft and empty when you load one.
- In the unlikely case that the topic is saved but the campaign cannot be, the topic stays, the administrator log records it as critical, and you can create the campaign from the topic as usual.

About the donation link

The link is rendered as a public button. Only `http://` and `https://` are accepted — `javascript:`, `data:` and protocol-relative addresses are rejected, and are also re-checked before rendering.

The extension does not visit the address or check that it works. Point it somewhere you control and have tested.

The button's wording is yours to choose per campaign, so it can describe what actually happens next — *Request bank details* when the link opens a page of instructions, *Über PayPal spenden* when it opens a payment page. The button is a link and nothing else: the extension embeds no provider form, loads no provider script, shows no logo and accepts no HTML. Clicking it takes the reader off the board, and you still confirm the money by hand and record it as a donation afterwards.

A link with no text would be a button with nothing written on it, so the extension refuses to save that combination rather than inventing a label.

⚠ About publishing payment details

Nothing here stops you putting your bank details in the campaign description, and people do. Understand what that means before you do it: the description is shown on a public topic that guests and search engines can read, and account identifiers published that way are routinely harvested and used in fraudulent payment requests.

If your board needs to publish payment details, treat it as an operational and privacy decision in its own right — not as a side effect of filling in a form. A donation link to a page you control is usually the safer shape.

5. Record a payment

Only after it has arrived and you have checked.

Recording happens **from the topic**, not the ACP. Open the topic, then **Topic tools** → **Donation campaign** to reach the management landing, and click **Add confirmed donation**. The donation ledger is shown only to an administrator or a holder of `f_donationcampaigns_donations` for that forum; a manage-only holder does not see it.

Field	Notes
Amount received	The amount that actually arrived. <code>50.00</code> or <code>50,00</code>
Payment received on	The date the money arrived , not today's date
Donor	The name to show publicly. Leave empty for <i>Anonymous</i>

Field	Notes
Show donor publicly	Untick to count the donation without naming the donor

Saving recalculates the campaign total immediately.

6. Decide whether to name the donor

Three settings interact:

1. **Campaign → Show donor names** — the master switch for that campaign.
2. **Donation → Show donor publicly** — per donation.
3. **An empty donor name** — always displayed as *Anonymous*.

A donation with the public flag off still counts towards the total and the donation count. Only the name is withheld.

Dates and anonymity. With *Show donation date* ticked, an anonymous donation is listed as "Anonymous — 50,00 € (3 Oct 2026)". In a small community, amount and day together can be enough to guess who it was. If that matters for a campaign, leave the date off.

Ask the donor before publishing their name. The extension cannot know whether you did. See [PRIVACY.md](#).

7. Edit or delete a confirmed entry

Donations are edited and deleted **from the topic**, from the same management landing, by an administrator or a holder of `f_donationcampaigns_donations` for that forum. Editing recalculates the total. So does deleting — the confirmation names the amount and donor so you can see what you are about to destroy.

Deleting a donation is permanent and there is no undo. A donation cannot be moved to another campaign; delete it and record it under the right one.

Deleting a campaign — the policy differs by where you do it:

- **From the topic**, only an *empty* campaign (no confirmed donations) can be deleted, by an administrator or a holder of `f_donationcampaigns_manage`. A non-empty campaign is refused there — disable it instead, or ask an administrator to delete it.
- **From the ACP**, an administrator (`a_donationcampaigns`) *may* hard-delete a *non-empty* campaign, cascading all of its donations. This is the deliberate difference: the ACP is the admin's tool for that case, and the confirmation names the campaign and its donation count.

Either way, deletion is permanent with no undo, and the total is always recomputed from the donation rows.

8. Recalculate a total

Campaigns → Recalculate total

The displayed total is a cached copy of the sum of that campaign's donations. It is recomputed on every add, edit and delete, so it should never drift. Use this action if you have edited the database directly, restored a partial backup, or simply want to confirm the figure — it recomputes from the donation rows and tells you the value before and after.

It is safe to run at any time and never changes a donation.

9. Disable or archive a campaign

Untick **Enabled** and save. The box disappears from the topic; every donation record is kept, and the campaign is still there on the topic, where you can edit it, add donations to it, or re-enable it from the management landing. The ACP lists it read-only.

Use this when a campaign has finished but you want the records.

9b. The board-wide campaign list

ACP → Extensions → Donation campaigns → Settings → Show the campaign list page. Off by default; an update never switches it on.

When it is on, the page `app.php/donationcampaigns` lists the campaigns, newest first, 25 per page, and the quick-links menu gains a **Donation campaigns** entry. Each row shows the campaign title (linked to its topic), the forum, the progress bar, collected / target / percent and — if the campaign shows it — the number of donations. **It never shows donor names**; those stay in the topic.

A visitor sees exactly the campaigns whose box they could see in the topic:

- the campaign is enabled;
- the topic exists and has not been moved away (a moved shadow is skipped);
- the visitor may read the forum;
- the topic is approved and not soft-deleted — moderators who may approve in that forum also see unapproved and soft-deleted topics, as in the forum;
- the forum has no password, or the visitor has already entered it in this session.

So guests see the campaigns in forums guests may read, and a campaign in a staff-only forum stays invisible to everyone else. Campaigns that have reached their target are listed like the others, with a full bar.

The quick-links entry is shown to everyone while the page is on, even to a visitor for whom the list is empty. prosilver hides the whole quick-links menu when quick links and search are both unavailable to a visitor; the page itself still works at its address.

9a. Where actions are logged

"Who did what, and where" is read from the log the entry lands in, which depends on the surface the action was performed on.

- **Actions taken from the topic** — creating, editing, enabling/disabling or deleting a campaign, and adding, editing or deleting a donation — are recorded in the **moderator log**, scoped to that forum and topic — also when the person acting is not a moderator. Read it in the ACP (**Maintenance → Moderator log**) or, as a moderator of the forum, in the MCP (**Moderator Control Panel → Forum logs**).
- **Actions taken in the ACP** — hard-deleting a campaign, recalculating a total, and changing settings — are recorded in the **administrator log** (**ACP → Maintenance → Logs**).

10. What happens when a topic or forum is deleted

This is the part worth knowing before someone tidies up the board.

Action	Effect on the campaign
Topic soft-deleted (moderator's "delete")	Nothing. The topic is recoverable, so the campaign and its donations are kept
Topic permanently deleted	The campaign and all its donations are permanently destroyed
Forum deleted with its content	Every campaign in it, and all their donations, are destroyed
Sub-forum deleted	Its campaigns are destroyed; the parent's are not
Topic pruned (manual, moderator, or scheduled)	Same as permanent deletion
All posts in a topic deleted	phpBB removes the emptied topic, so the campaign goes too
User deleted with their posts	Their topics are removed, and campaigns on those topics go with them

Action	Effect on the campaign
Topic moved	Nothing. The campaign follows the topic

Deletion is atomic: either the campaign and all its donations go, or nothing does. No orphaned donation rows are left behind.

There is no warning and no undo. phpBB does not know it is about to destroy a financial record. If a campaign's history matters, export it before deleting its topic — and prefer *disabling* the campaign to deleting the topic.

Purging the extension destroys everything it stores, in every campaign.

Troubleshooting

The box does not appear on the topic. Check the campaign is enabled and purge the cache. The box renders on `viewtopic`; if the topic itself is unreachable, so is the box. If the box, the *Manage* button **and** the topic-tools entry are all missing while the ACP pages work, the `styles` folder did not reach the server — see *Updating to a newer version* in step 1.

The total looks wrong. Use **Recalculate total**. If it changes, something wrote to the database outside the extension.

Amounts are out by a factor of ten or a hundred. The decimal-places setting was changed after the data was recorded. See step 3.

A donor's name appears when it should not. Check both switches: the campaign's *Show donor names* and that donation's *Show donor publicly*.

The "Donation campaign" tab is missing from the posting form. It appears only for a new topic, and only in forums where the user may manage campaigns and read the forum (step 2). It is never shown when replying or editing.

The campaign list says the page could not be found. The list is switched off (the default) — see step 9b. After updating the files, purge the cache before testing.

A campaign is missing from the campaign list. Check the five conditions in step 9b; the most common causes are a disabled campaign, a forum the visitor may not read, and a password-protected forum not yet entered.

Menu entries have vanished. The extension is disabled. Data is intact.

Privacy and data handling

What this extension stores, what it never stores, and what publishing a donor's name actually does.

This document describes **technical behaviour**. It is not legal advice and makes no claim of compliance with GDPR or any other regime. Whether your use of this extension is lawful is a question for you and, if the answer matters, your own advisers.

See also: [administrator guide](#) · [README](#)

What is stored

Two tables, and nothing else.

Campaigns — `phpbb_ufdc_campaigns`

Column	What it holds
<code>campaign_id</code>	Internal identifier
<code>topic_id</code>	The phpBB topic the campaign is attached to
<code>campaign_title</code>	Title, as the administrator typed it
<code>campaign_desc</code>	Description, plus three columns of phpBB BBCode metadata
<code>target_amount</code>	Target, as a whole number of the smallest currency unit
<code>collected_amount</code>	Cached sum of the campaign's donations
<code>campaign_enabled</code>	Whether the public box is shown
<code>show_donor_names</code>	Whether donor names may be listed
<code>show_donation_count</code>	Whether the number of donations is shown
<code>show_donation_date</code>	Whether the donor list shows the day each donation arrived
<code>external_url</code>	Optional donation link
<code>campaign_created</code> , <code>campaign_updated</code>	Timestamps

Donations — `phpbb_ufdc_donations`

Column	What it holds
<code>donation_id</code>	Internal identifier
<code>campaign_id</code>	The campaign it belongs to
<code>donation_amount</code>	Amount, as a whole number of the smallest currency unit
<code>donor_name</code>	A free-text display name, typed by the administrator
<code>donation_time</code>	The date the payment was received
<code>donation_public</code>	Whether the name may be shown
<code>donation_created</code> , <code>donation_updated</code>	Timestamps

Four board settings are also stored: currency code, symbol, decimal places, and how many donors the public box lists.

What is not stored

None of the following exists anywhere in this extension — not in a column, not in a log, not in a cache:

- Bank account numbers, IBANs, BICs, sort codes
- Payment credentials of any kind
- PayPal, Stripe or any other transaction or reference identifier
- Payment-provider callbacks or webhook payloads
- Card numbers or any card data
- Proof-of-payment files, screenshots or attachments
- Donor email addresses, postal addresses or telephone numbers
- Donor IP addresses
- Any link between a donation and a phpBB user account
- Private internal notes — there is no notes field

A donation record is an **amount, a date, and a name someone typed**. That is the whole of it.

The extension makes **no outbound network requests**. It never contacts a payment provider, an analytics service, or any third party. Nothing is sent anywhere.

⚠ These guarantees describe the extension's own fields. An administrator can type anything into the campaign description or a donor name, including data that does not belong on a public page. The extension cannot prevent that — see below.

The donor name is the sensitive part

Everything else here is bookkeeping. The donor name is the one field that can identify a living person, and the extension makes publishing it a single checkbox. That deserves stating plainly:

The extension can publish a real person's name on a page that guests can read and search engines can index. It cannot know whether that person agreed. You hold that responsibility, not the software.

The name is free text: it can be a first name, an initial, a pseudonym, a company, or a full legal name. What appears is exactly what was typed.

What "public" means

A campaign box appears on a normal topic. If guests can read that forum, then the donor list is visible to anyone on the internet and can be crawled, cached and archived by search engines. Removing a name later does not remove it from caches and archives that already hold it.

The three controls

1. **Leave the donor name empty.** The donation is recorded and counted, and the box shows *Anonymous*. Nothing identifying is stored at all.
2. **Untick "Show donor publicly"** on the donation. The name is stored but never rendered publicly. The donation still counts towards the total and the donation count.
3. **Untick "Show donor names"** on the campaign. No names are listed for that campaign, whatever the individual donations say.

A fourth setting affects how identifiable an *anonymous* donation is: **"Show donation date"** on the campaign. With it ticked, the list shows the day next to every amount, including anonymous ones. Amount plus day can point to a person in a small community; leave it unticked where that is a concern. It is off for every campaign created before 1.0.0-beta2.

A private donation is not hidden from the *figures* — its amount is included in the total and in the count. Only the name is withheld. If a donor must not be counted at all, do not record the donation.

Before you publish a name

Ask the donor. The safe default is to record donations anonymously and add a name only when someone has actually asked to be named.

Deletion and erasure

Request	What to do
"Remove my name, keep the donation"	Edit the donation, clear the donor name. It becomes <i>Anonymous</i> ; the total is unchanged
"Do not show my name publicly"	Edit the donation, untick <i>Show donor publicly</i> . The name is retained but never rendered
"Delete my donation entirely"	Delete the donation. The campaign total is recalculated immediately

Clearing the name is usually what people want: the financial record stays accurate and nothing identifying remains.

Deletion is immediate and permanent. There is no archive, no soft-delete and no undo for donation records.

Cascade behaviour

Campaign and donation data is destroyed automatically when the topic it is attached to disappears:

Event	Effect
Topic soft-deleted	Data kept — the topic is recoverable
Topic permanently deleted , or pruned	Campaign and all its donations destroyed
Forum deleted with its content	Every campaign in it destroyed, with all donations
Sub-forum deleted	That sub-forum's campaigns destroyed
All posts in a topic deleted	phpBB removes the topic, so the campaign goes too
User deleted with their posts	Their topics go, and campaigns on those topics with them

Deletion is atomic — donations are removed before their campaign, so no donation row is ever left pointing at a campaign that no longer exists.

Note that deleting a phpBB **user** does not by itself remove donations naming that person: the donor name is free text with no link to an account. Handle those separately.

Uninstalling

- **Disable** — data is kept and nothing is shown. Reversible.
- **Purge** — every campaign, every donation, the settings and the permission are destroyed. Not reversible.

Back up before purging.

Logging

Recording, editing and deleting a campaign or donation writes an entry to phpBB's **admin log**, which administrators can read. Those entries include the campaign title, or the donation amount and donor name.

That means a donor name can survive in the admin log after the donation itself is deleted. If an erasure request has to reach the log too, clear it through **ACP** → **Maintenance** → **Admin log**; the extension does not manage that log's retention.

No logging happens on the public side. Viewing a topic with a campaign records nothing.

Donation Campaigns — Release Notes

1.0.0-beta4

Status: Fourth public beta, a technical one. The templates are now written in native Twig, a few leftovers of older phpBB coding patterns are gone, and one defect from beta3 is fixed. Nothing changes for visitors, managers or administrators: the pages look and behave exactly as in beta3.

Requires phpBB **3.3.16 or later** (below 4.0). prosilver only. PHP **8.2 or newer**.

Changed: templates in native Twig

All fifteen templates use Twig syntax — `{{ VAR|e }}`, `{% if %}`, `{% for %}`, `{{ lang('KEY') }}` — instead of phpBB's legacy syntax (`{VAR}`, `<!-- IF -->`, `<!-- BEGIN -->`, `{L_KEY}`). The occasion was a note from the Extension Check team of phpBB.de; phpBB's documentation ("Tutorial: Template syntax") says the legacy syntax will be deprecated and recommends Twig, and the official Skeleton Extension writes Twig only.

phpBB still understands the legacy syntax, so this changes spelling, not behaviour. Each step was checked against the previous version: the compiled templates and 46 captured page states of a test board are identical. A test keeps the legacy syntax out. See ADR-020 in [DEVELOPERS.md](#).

Also changed, none of it visible:

- Developer notes in the templates are Twig comments now, so they no longer appear in the page source.
 - Input tags end without a space before `>` .
 - The frontend controllers receive phpBB's PHP file extension from the service container instead of reading a global variable.
 - The stylesheet uses the standard property name `overflow-wrap` instead of the older `word-wrap` .
-

Fixed

- **Since beta3, the post text appeared a second time in the page source of the posting form.** When a manager started a new topic and the form was shown again — after a preview or with an error message — the text of the post was repeated inside an HTML comment in the page source. It was not visible on the page and could not be used to inject anything (the text is escaped), but it did not belong there. The cause was a developer comment that named phpBB's post-text variable, which phpBB's template engine filled in even inside a comment. Developer notes are no longer HTML comments, and a test renders the form with a marker text and fails if it appears.
-

Upgrade from 1.0.0-beta3

1. Disable the extension (not "Delete data"), upload the new files over the old ones and check that all twelve folders arrived — see *Updating to a newer version* in the administrator guide. Then enable it again.
2. **Purge the cache** (ACP → General → Purge the cache). Boards that do not recompile changed templates automatically keep serving the old ones until then.
3. There is **no migration** in this release: no database change, no new setting, no permission change.

Updating straight from an earlier beta: read the beta3 and beta2 notes first ([RELEASE NOTES BETA3.md](#), [RELEASE NOTES BETA2.md](#)).

For translators

No new, changed or removed language keys. The language files are unchanged since beta3; existing translations work as they are.

Developer notes

Architecture, the contracts that hold it together, and how it is tested.

See also: [README](#) · [administrator guide](#) · [privacy](#)

Package

Composer name	uflagmey/donationcampaigns
Directory	ext/uflagmey/donationcampaigns/
Namespace	uflagmey\donationcampaigns
Licence	GPL-2.0-only

Identifiers that enter a shared phpBB namespace carry the full `donationcampaigns` prefix — config keys, the permission, service ids, language keys, template variables, form keys, log keys, CSS classes.

Physical database identifiers are the one exception, using the abbreviated stem `ufdc`: `phpbb_ufdc_campaigns`, `phpbb_ufdc_donations`. Oracle caps identifiers at 30 bytes, phpBB 3.3 supports Oracle, and phpBB validates column and index names but *not* table names — an over-long table name fails as a raw driver error during installation. `phpbb_donationcampaigns_campaigns` is 33 bytes. A test asserts every physical identifier still fits.

Layers

ACP module / event listeners	coordinate only — no SQL, no rules
↓	
services	business rules, transaction boundaries
↓	
repositories	persistence only — all SQL lives here

- **Repositories** hold every query. They cast values to their intended PHP types at the boundary, so no consumer has to remember the database returns strings. Not-found contract: single reads return `null`, list reads return an empty array, counts and sums return `int 0`.
- **Services** own validation, ordering and transactions. They take the database handle *only* to open transactions and issue no SQL of their own.
- **Listeners and the ACP module** read input, call a service, and render.

A test suite reads the source and fails the build if SQL appears outside a repository, or if the database handle is used for anything but a transaction.

Money

Every amount is an **integer number of the smallest currency unit**. 10.00 EUR is `1000`. No float, no double, no `* 100`, no `/ 100`, anywhere.

Parsing and formatting are string operations in `currency_formatter`, because the obvious implementation loses money on ordinary input: `(int) ('8.70' * 100)` evaluates to `869`. The test table includes that case, and a 2001-iteration round-trip proves parse/format is lossless.

An architectural test forbids `(float)`, `(double)`, `floatval`, `round()` and `number_format` in production code.

The campaign total

`campaigns.collected_amount` is a **cache**. The donation rows are the truth.

After every donation create, edit and delete, the total is recomputed as `SUM(donation_amount)` and written inside the same transaction. It is never adjusted by arithmetic — no `total + amount`, no `total - amount`, no `total + (new - old)`.

Deltas compound: one missed rollback or one row changed out of band and the figure is permanently wrong with nothing able to detect it. Recomputing is self-healing — any single successful mutation repairs whatever drift preceded it. Four tests seed a deliberately corrupted total and assert that a subsequent add, edit, delete or explicit recalculation fixes it, and a 200-operation randomised sequence asserts, after every committed operation, that every campaign's stored total equals a freshly computed `SUM()`.

`collected_amount` is absent from the services' writable-field lists, so it can never arrive from request input.

Transaction boundaries

Method	Boundary
All reads, and validation	None
<code>campaign_service::create_campaign()</code> / <code>update_campaign()</code>	None — one statement
<code>campaign_service::delete_campaign()</code>	Own
<code>campaign_service::purge_for_topics()</code>	Own; none at all when nothing matches
<code>campaign_service::purge_for_forum()</code>	Delegates to <code>purge_for_topics()</code>
<code>donation_service::*</code> (add, edit, delete, recalculate)	Own, via one private helper

phpBB's `sql_transaction('rollback')` is **not** nest-aware: it unwinds the whole active transaction, not just the extension's level. That is deliberate here — a cleanup failure inside core's deletion aborts core's deletion too, rather than leaving orphans.

Confirmed-donation semantics

A donation row is money **already received and verified by an administrator**. The extension processes no payments, contacts no provider, makes no outbound requests, and exposes no public entry point. Version 1.0 supports manually confirmed donations only.

The action button

`external_url` is a destination and `external_link_text` is the label on the button pointing at it. Both are per campaign, and neither carries provider semantics: there is no provider column, no enum, no transaction id, no callback and no embedded markup. A PayPal link and a link to a forum topic explaining bank transfers are the same thing to this code.

The label is plain text with **no BBCode pipeline** — unlike `campaign_desc`, nothing parses it — stored raw and escaped once at output with `|e`. The column is `VCHAR:100`, deliberately narrower than the 255 used elsewhere in the table, because a button label past that length renders badly at any width; `campaign_service::MAX_LINK_TEXT_LENGTH` matches it and a test asserts the two stay equal. Length is counted in characters, not bytes.

A URL with an empty label is refused by `campaign_service` rather than substituted at render time. Falling back silently would accept a submission the administrator did not mean to make; with no URL the button is not rendered at all, so an empty label is simply unused and permitted.

Decision: no payment-provider integration

The extension intentionally treats payment destinations as ordinary external HTTPS links. No provider-specific integration exists in version 1.0.

This was decided deliberately, not deferred for lack of time. A per-campaign URL plus a per-campaign button label already expresses every destination a board needs, and it does so without the extension knowing what any of them are. PayPal works today: paste the hosted donation link from a PayPal account into the campaign URL and set the label to say so.

A board-wide provider mode was designed and **rejected**. Two reasons decided it. A global setting that supplied the destination would override a campaign that had deliberately been pointed at a bank-information topic, which breaks boards running more than one kind of destination at once. And generating a provider URL from a merchant identifier means adopting that provider's link format — the obvious PayPal one belongs to a deprecated integration, and the current replacement requires provider-hosted setup and external JavaScript, both outside what this extension will load.

Deferred future work, explicitly not planned for 1.0: if a provider ever genuinely needs automation, phpBB's `service_collection` tagging is the mechanism (core selects authentication providers that way in `provider_collection::get_provider()`). Nothing in the current schema blocks it, because campaigns store a destination and a label and never a provider. Until such a provider exists, no registry, interface or abstraction should be added — one implementation behind an interface is not extensibility, it is indirection.

Deletion cascades

Two listeners, because phpBB destroys topics two different ways.

Event	Covers
<code>core.delete_topics_before_query</code>	Topic deletion, ACP/MCP/scheduled pruning, deleting the last post of a topic, deleting a user with their posts
<code>core.delete_forum_content_before_query</code>	Forum and sub-forum deletion

`prune()`, `auto_prune()`, `delete_posts()` (when it empties a topic) and `user_delete()` all funnel into `delete_topics()`, so one listener covers them all. **Forum deletion is the exception** — it removes topics with a direct `DELETE ... WHERE forum_id` and never calls `delete_topics()`. A test asserts that asymmetry so it fails loudly if core ever changes.

⚠ The forum event's `topic_ids` payload is **not** usable. Core builds it from a join against the attachments table, so it lists only topics that *have* an attachment, repeats a topic once per attachment, and omits ordinary topics entirely. The service resolves the real topic list through `topic_repository` instead. The test fixture reproduces that misleading payload deliberately, so an implementation trusting it fails.

Donations are always deleted before campaigns. Donation rows key on `campaign_id`; reverse the order and the ids become unresolvable and the rows are orphaned permanently. Asserted on the statements actually issued, on both SQLite and MariaDB.

There is deliberately **no denormalised `forum_id`** on campaigns — it would make the cascade trivial and then have to be kept correct on every topic move, forever.

⚠ Known defect: merging or fully splitting a topic destroys its campaign

Release-blocking; a fix is required before 1.0 final. Recorded here rather than left to be rediscovered.

Merging topics, or splitting every post out of one, empties the source topic. `move_posts()` then calls `sync('topic', ...)`, which collects the now-empty topic into `$delete_topics` (`functions_admin.php:1967`) and calls `delete_topics()` (`:2058`). That fires `core.delete_topics_before_query` (`:833`) — the event this extension's cascade listens to — and the campaign and **every confirmed donation record attached to it** are deleted.

Three things make this worse than it first sounds: the deletion is silent, it is irreversible, and it can be triggered by a moderator who does not hold `a_donationcampaigns` and gets no indication that anything was lost. These are records of money actually

received.

The cascade itself is not wrong — an orphaned campaign pointing at a dead topic would be worse. The fix belongs at the level of *distinguishing* a topic being destroyed from a topic being emptied into another one, and must be designed rather than patched. It must **not** be addressed by reintroducing a context-free creation form so the campaign can be retyped.

Shadow topics

A campaign cannot attach to the stub left behind by a moved topic. `viewtopic.php?t=<shadow>` answers HTTP 404 "*The requested topic does not exist*" — identical to a topic that never existed — so a campaign there would render nowhere. `topic_repository::topic_exists()` excludes `topic_moved_id != 0`, and validation reports it exactly as a missing topic.

Cleanup deliberately still finds shadows, so a campaign attached to one before this rule is still purged with its forum rather than becoming both unreachable and undeletable.

phpBB integration points

Verified against core source, not inferred from names.

Point	Location	Note
<code>core.viewtopic_assign_template_vars_before</code>	<code>viewtopic.php:775</code>	<code>topic_id</code> in scope, after access checks
<code>viewtopic_body_poll_before</code> (template)	<code>viewtopic_body.html:77</code>	Despite the name, sits outside the <code>S_HAS_POLL</code> conditional that opens two lines later, so it renders on every topic
<code>core.delete_topics_before_query</code>	<code>functions_admin.php:833</code>	Inside core's transaction, opened at 817
<code>core.delete_forum_content_before_query</code>	<code>acp_forums.php:2011</code>	Before core's <code>DELETE ... WHERE forum_id</code> at 2013
<code>viewtopic_topic_tools_after</code> (template)	<code>viewtopic_topic_tools.html:46</code>	Last item in the wrench dropdown. That file is <code>INCLUDE</code> d twice (<code>viewtopic_body.html:44</code> and <code>:418</code>), so anything in it renders in both action bars — the entry therefore carries no <code>id</code> attribute
<code>S_DISPLAY_TOPIC_TOOLS</code>	<code>viewtopic_topic_tools.html:1</code>	Referenced by the wrapper condition and assigned nowhere in core PHP . It exists so an extension can force the dropdown open when no core tool would have
<code>core.permissions</code>	—	Declares the three permissions (<code>a_donationcampaigns</code> plus the two forum-scoped <code>f_donationcampaigns_*</code> , ADR-016) and their category to the ACP UI

Point	Location	Note
<code>overall_header_head_append</code> (template)	—	<code>INCLUDECSS</code> for the stylesheet
<code>core.page_header</code>	<code>functions.php:3845</code>	Quick-links entry for the board-wide list (ADR-018); returns at once while the ACP switch is off
<code>navbar_header_quick_links_after</code> (template)	<code>navbar_header.html:75</code>	Last item of the quick-links menu. prosilver hides the whole menu when quick links and search are both off for the viewer, and the entry with it
<code>core.posting_modify_template_vars</code>	<code>posting.php:2089</code>	Posting panel values (ADR-019); only <code>mode = post</code> with the manage permission
<code>core.posting_modify_submission_errors</code>	<code>posting.php:1428</code>	Field validation on submit and preview; <code>error</code> takes sentences, not keys
<code>core.posting_modify_submit_post_after</code>	<code>posting.php:1603</code>	Campaign creation; the new id is in <code>data['topic_id']</code> (<code>functions_posting.php:2021</code>), the event's <code>topic_id</code> is still 0; runs before the approval branch (<code>posting.php:1619</code>)
<code>posting_editor_add_panel_tab</code> (template)	<code>posting_editor.html:134</code>	The tab, after the poll tab; same markup, so <code>forum_fn.js</code> switches it
<code>posting_layout_include_panel_body</code> (template)	<code>posting_layout.html:59</code>	The panel, after the poll panel
<code>content_visibility::get_forums_visibility_sql()</code>	<code>content_visibility.php:250</code>	Topic approval / soft-delete SQL for the board list. Intersects the <code>m_approve</code> forums with the given list only if that list is non-empty — never call it with an empty list (ADR-018)

The escaping contract

phpBB 3.3 runs Twig with **autoescape disabled** (`phpbb/template/twig/environment.php:79`), so nothing is escaped for you.

Domain scalars are stored raw — campaign titles, donor names, external URLs, currency code and symbol. They are read with `$request->raw_variable()`, never `variable()`, which would escape on input and store `&` for an administrator who typed `&`.

Core-owned titles are the exception: they are stored escaped. phpBB reads a topic subject and a forum name through `request->variable()`, so `topic_title` and `forum_name` sit in the database as `Kosten & Miete`. Core's templates print them without `|e`, and so do ours. Until beta3 three templates added `|e` and showed `Kosten & Miete` (fixed in 1.0.0-beta3). `architecture_test::CORE_ESCAPED_FIELDS` names these fields; a template that escapes them fails the build. The reverse also occurs: the posting form turns core's escaped subject into a raw campaign title with `htmlspecialchars_decode(..., ENT_COMPAT)`, the exact inverse of core's encoding (ADR-019).

Escaping happens at output, in exactly two places:

1. **Templates.** Every administrator-controlled scalar carries Twig's `|e : {{ VAR|e }}`. Templates are native Twig (ADR-020). No `|raw`, ever, and global autoescape is never enabled.
2. **Sinks with no template.** `confirm_box()` renders `{MESSAGE_TEXT}` raw, and the ACP log viewer `sprintf`'s log parameters into a language string and prints the result unescaped (`phpbb/log/log.php:665-710`, rendered by `{log.ACTION}`). Both are escaped immediately before the message is built, with phpBB's `utf8_htmlspecialchars()`. The escaped value is never written back to the database.

`U_*` URL variables are assigned already attribute-safe and must **not** carry `|e`, or they would be escaped twice.

A test forbids any direct `htmlspecialchars()` call in production code, requires `|e` on every administrator-controlled scalar in every template, and forbids it on the core-escaped fields. These rules read each template as phpBB's lexer hands it to Twig (`template_renderer::lexed()`), and each also asserts that it found something to check; the rendering tests show the escaped result on the page (see *Tests*).

The BBCode description

The campaign description is the one formatted field and takes the **opposite** contract:

input	<code>\$request->variable(..., true)</code>	escapes
	<code>description_formatter::for_storage()</code>	phpBB's <code>generate_text_for_storage()</code>
storage	<code>text + uid + bitfield + flags</code>	stored together; meaningless apart
display	<code>for_storage's result → for_display()</code>	rendered with NO <code> e</code>
edit	<code>for_edit()</code>	decoded back to source for the textarea

`generate_text_for_display()` does **not** sanitise — it censors and parses BBCode and assumes the text was escaped on the way in. Escaping its output again would render an administrator's formatting as visible tags.

Encoding lives in `campaign_service`, not the ACP module, so no caller can store a raw description. The BBCode metadata columns are absent from the writable fields: they are produced by the encoder, never accepted from a request, because a uid describing markup the text does not contain is how a payload gets past the display path.

`description_formatter` is a thin seam over phpBB's three content functions, which resolve `text_formatter.parser` from the container and therefore need a booted board. Unit tests substitute a fake and assert the service's *policy*; the real pipeline is exercised on the Docker board.

phpBB 3.2+ stores BBCode as **s9e XML** (`<r>...<B><s>[b]</s>...</B></r>`) and leaves `desc_bbcode_uid` and `desc_bbcode_bitfield` empty. That is correct, not a bug; the columns are still required by the display and edit functions.

ADR-014 — Campaigns are created only from their topic, and the ACP resolves state at request time

Superseded by ADR-015. Management no longer lives in the ACP: campaign and donation actions moved to frontend controllers reached from the topic, under forum-scoped moderator permissions. ADR-014 is kept as the historical record of the topic-context creation decision, whose reasoning ADR-015 carries forward.

Creation used to mean reading a topic's numeric id out of its address and typing it into the ACP. Nothing else in phpBB asks that, and the field it was typed into was the one place a campaign could be pointed at the wrong topic.

The entry point is now a single neutral **Donation campaign** item in the topic tools menu. It carries no verb, and the URL carries no action — only the topic. The ACP resolves what to do when the request arrives:

no campaign for this topic	-> create
a campaign, enabled or disabled	-> edit that campaign

Why neutral rather than "Add" / "Manage". A topic page is rendered once and may be clicked much later. Any verb decided at render time is a claim about the past: a campaign can be created, deleted or disabled in between. Resolving on arrival makes stale pages, disabled campaigns, concurrent creation and hand-edited URLs one mechanism instead of four guards. It also keeps the listener free of a campaign lookup, which matters because it runs on every topic view on the board.

Topic context suppresses the action parameter entirely. With a topic in the URL, `action` is not read at all, so `t=10&action=delete&campaign_id=1` cannot be expressed rather than being caught.

The expected-state guard. Resolving on arrival alone would silently turn a lost create race into an edit, overwriting the campaign that won with values typed for one that did not exist. The form records which campaign it was drawn for; on a mismatch nothing is written and the form is redrawn with an explanation. The recorded id is untrusted but can only ever *downgrade* a write to a re-render — the campaign written is always the resolved one.

Two permissions. `a_` is a real ACL option, not a prefix wildcard (`schema_data.sql:411`), so an administrator holding `a_donationcampaigns` without `a_` is refused by `adm/index.php` with a 403. Both are checked before the link renders, and both checks are visibility only — `main_module::main()` still enforces access itself.

The module identifier is derived, never written. phpBB maps a class name to a URL token by replacing backslashes with dashes (`functions_module.php:1141–1150`). Since this link is the only route to creation, a stale literal would remove the feature rather than degrade it, and would fail only for administrators, only at runtime. Two tests cover it.

Links out of the ACP must carry `$phpbb_root_path`. This code runs inside `adm/`, where a bare `viewtopic.php` resolves to `/adm/viewtopic.php`. Every unit test asserting "the URL contains `t=30`" passes either way; only following the link on a running board shows it. All such links go through one helper and a regression test asserts the root path is present.

Consequence, accepted deliberately. A style that does not provide `viewtopic_topic_tools_after` offers no way to create a campaign at all. Under ADR-013 that style is outside v1.0's supported presentation scope. A weaker ACP form was explicitly rejected as a fallback, because it would have preserved the raw topic-id workflow this decision exists to remove.

ADR-015 — Frontend-primary management with forum-scoped moderator permissions

Permission model superseded by ADR-016. The frontend-primary decision, the layering, the deletion policy, audit by surface and uniform denial below all still stand. Only the *type* of the two forum-scoped permissions changed: they are forum permissions (`f_`) since 1.0.0-beta2, not moderator permissions (`m_`).

Campaign **and** donation management moved out of the ACP into frontend controllers reached from the topic. The topic-tools **Donation campaign** entry now opens a management landing (`/app.php/donationcampaigns/topic/{topic_id}`) that resolves state on arrival — the create form when there is no campaign and you may make one, otherwise a campaign summary plus only the actions you are authorised for. This supersedes ADR-014.

Why. A public extension needs *forum moderators*, not only board administrators, to run donations — the person who manages a fundraising topic is usually its moderator, not someone with ACP access. The old model could not express that: it had a single global `a_donationcampaigns` and did every write in the ACP, so the only way to let someone manage donations was to make them an administrator of the whole board. The ACP-only shape also did not feel like native phpBB, where topic-level work happens on the topic.

The three-permission model, forum-scoped. One global admin permission plus two forum-scoped moderator permissions:

- `a_donationcampaigns` — global admin, granted to the admin roles on install. Full ACP access and an override for every frontend action on every forum.
- `m_donationcampaigns_manage` — forum-scoped. The campaign shell: create, edit, enable/disable, delete an *empty* campaign.
- `m_donationcampaigns_donations` — forum-scoped. The donation ledger: add, edit and delete confirmed donations.

The two `m_` permissions are independent and neither is granted on install; `a_donationcampaigns` overrides both. All three sit in a dedicated **Donation Campaigns** permission category. Authorisation is checked per forum, resolved from the campaign's topic.

The layering did not change. `Repository → Service → Controller` mirrors the old `Repository → Service → ACP module`: the service layer was left untouched. The frontend controllers are thin coordinators that read input, call the same services and render, exactly as the ACP module did. No business rule moved into a controller.

Deletion policy is split deliberately. From the topic only an *empty* campaign may be deleted; a non-empty one is refused, to protect records of money received from a forum moderator's cleanup. The ACP keeps an admin-only hard-delete that cascades a *non-empty* campaign's donations — that is the administrator's tool for the case the frontend refuses. Donations are deleted individually behind a confirmation naming the amount and donor.

Audit is by surface. Frontend actions (campaign create/edit/enable/disable/ delete; donation add/edit/delete) are written to the **moderator log**, scoped to the forum and topic, so they surface in the MCP alongside other moderator activity. ACP actions (hard-delete, recalculate, settings) are written to the **administrator log**. The log an entry lands in identifies where the action was performed.

Denial is a uniform 404. An unauthorised or malformed request — a forum the caller cannot manage, a missing topic or campaign, a mismatched id — answers with the same "not found" response rather than distinguishing "forbidden" from "does not exist", so the frontend never discloses whether a given campaign or donation exists to someone not entitled to know.

The ACP became read-only oversight plus admin maintenance. It keeps settings, a read-only campaign list and a read-only donation list, and the admin-only maintenance (recalculate a total, hard-delete a non-empty campaign). Its former create/edit forms are gone; the list rows link out to the frontend routes on the topic.

Consequence, accepted deliberately — and later revisited. phpBB treats any holder of an `m_` permission on a forum as a moderator of it, so granting either donation permission lists the grantee among that forum's moderators and gives them moderator standing there. This is inherent to phpBB's definition of a moderator and was documented for board owners as a real (if limited) promotion, not a hidden grant. ADR-016 removes this consequence.

ADR-016 — Forum permissions (`f_`) instead of moderator permissions

Decision (1.0.0-beta2). `m_donationcampaigns_manage` and `m_donationcampaigns_donations` are replaced by `f_donationcampaigns_manage` and `f_donationcampaigns_donations`: local, independent, granted to nothing on install, same meaning as before. `a_donationcampaigns` and its override are unchanged. Every frontend check additionally requires `f_read` on the forum.

Why. Review feedback from the phpBB.com community: permissions should be assignable to any group, so a board can let moderators *or* non-moderators run campaigns. The `m_` options were technically assignable to any group, but in phpBB an `m_` grant is never only itself. Verified against 3.3.17:

- `auth_admin::acl_set()` (`includes/acp/auth.php:843–858`) also sets the `m_` any-flag;
- that flag opens the MCP (`mcp.php:107` , `acl_getf_global('m_')`) and drives the MCP link, topic logs and topic-type change on `viewtopic` (`viewtopic.php:637, 668, 793`);
- `phpbb_cache_moderators()` collects every `m_%` grant, so the holder is listed as a moderator of the forum and in "The team".

Forum permissions have none of these effects. They are the family phpBB uses for feature rights inside a forum — `f_poll` is the direct analogue — and they are assigned per group and per forum on the Forum permissions tab. They also fit the planned posting-form integration (creating a campaign while creating the topic), which `posting.php` gates on forum permissions.

Read access as a precondition. With forum roles a board can easily end up with "may manage campaigns" but not "may read the forum". The access service closes that once, for everyone including the administrator override: nobody manages from the frontend a topic they could not open.

Upgrade: clean break, with the orphaned flag removed. `m8_forum_permissions` does not carry beta1 grants over; the board owner re-assigns the new permissions (beta, small install base, owner decision). The migration must still undo the moderator standing the old grants created: `permission.remove` deletes only the rows of the option it removes, never the `m_` any-flag written alongside, so without help every former holder would keep MCP access and stay listed as a moderator. `clear_orphaned_flags()` deletes that YES flag for each group, user and role whose only YES moderator options were ours; it runs before the removal, while the grants still show who held them, which keeps every step stateless. The moderator cache is

rebuilt last. `m7` is left untouched as a shipped migration; `m8`'s revert re-creates the `m_` options ungranted so `m7`'s own revert still finds them.

Unchanged on purpose. Frontend actions still log to the **moderator log** (audit by surface, ADR-015), also when the actor is not a moderator; the entry is filed against forum and topic and is readable in the ACP moderator log and in the MCP by the forum's moderators.

ADR-017 — One place for money display, dates as calendar days

Decision (1.0.0-beta2). `currency_formatter::format_money()` is the only function that attaches the currency symbol to a number. Every displayed amount — topic box, donor list, management landing and ledger, ACP lists, confirmation dialogs, log entries — calls it. `format()` (number only) remains for parse round-trips; `format_for_input()` for form fields, whose symbol label sits on the configured side of the input. Two board settings control the output:

`donationcampaigns_currency_symbol_before` and `donationcampaigns_currency_symbol_space`; the separator is U+00A0, so an amount never wraps between number and symbol.

Why. Review feedback asked for the symbol before or after the amount. The analysis showed the symbol was appended in one place only (the topic box), so every other page showed bare numbers — seen live on the management landing. A position setting bolted onto that one place would have widened the inconsistency. An architecture test now fails on any `formatter->format()` in production code.

Donation dates. `donation_time` is a calendar day stored as midnight UTC. `date_formatter` renders it in UTC through `phpbb\datetime` (translated month names) with a translator-owned format, `DONATIONCAMPAIGNS_DATE_FORMAT`. The previous ACP rendering through `user::format_date()` used the viewer's time zone and a time of day, so a viewer west of UTC saw the previous day; that is fixed with the same service.

Public dates are opt-in per campaign (`show_donation_date`, `m9`). Existing campaigns keep the column default 0, so an update changes no public page; the create form proposes it ticked for new campaigns. A date is only meaningful in the donor list, so it shows only where names are shown. The listener does not assign the key at all when the campaign has not opted in.

Manage button. The box header carries a "Manage" button bound to the same flag and URL as the topic-tools entry (`S_DONATIONCAMPAIGNS_TOPIC_LINK`), so its visibility is exactly the access rule and no second rule exists.

ADR-018 — A third, read-only controller for the board-wide campaign list

Decision (1.0.0-beta3). `controller/list_controller.php` serves `app.php/donationcampaigns` (route `uflagmey_donationcampaigns_list`, GET only): every campaign the viewer may see, 25 per page, newest first, with title (linked to the topic), forum, the shared progress bar, collected / target / percent and, if the campaign shows it, the donation count. No donor names. A board setting, `donationcampaigns_list_enabled` (`m10`, **off by default**), switches the page and its quick-links entry on; while it is off — or before `m10` has run — the route answers 404 `PAGE_NOT_FOUND`.

Why a third controller. The phpBB.com review asked for a site-wide list with an ACP switch. A public, paginated page needs a route, and a route needs a controller; phpBB 3.3 offers no event-only way to render a standalone page. The rule "avoid additional controllers or duplicated write paths" exists to keep a single write path per action. This controller is not a write path: one action, no form, no form key, no write method. `architecture_test` fails the build if a write method, a form key, POST handling or a log call appears in it. Alternatives: a block on the board index (no pagination, clutters the index, not what was asked) and the ACP list (exists, not public).

Visibility — the part that must be right. A campaign is listed only if all hold, i.e. exactly when its box would be visible in the topic:

1. `campaign_enabled = 1`;
2. its topic exists and is not a moved shadow (`topic_moved_id = 0`);
3. the viewer has `f_read` on the topic's forum;
4. the topic is visible to the viewer per core's `content_visibility` (unapproved / soft-deleted topics only for `m_approve`);
5. the forum has no password, or this session has unlocked it — otherwise viewtopic shows the forum login box instead of the topic.

The filter runs in SQL (one JOIN of campaigns, topics and forums), not in PHP after fetching, so the page count matches the pages. `campaign_list_service` builds the forum list (rules 3 and 5, from `acl_getf('f_read')` and `user::get_passworded_forums()`) and core's visibility fragment (rule 4); `campaign_list_repository` applies them together with rules 1 and 2. When no forum is left, the service answers "nothing" **before** building the fragment: `get_forums_visibility_sql()` with an empty forum list would open every `m_approve` forum regardless of `f_read`. The repository applies the forum list itself as well, so it never relies on the fragment alone.

Read model, not new methods on the existing classes. `campaign_repository` is single-table by design and `campaign_service` carries no authorisation (that lives in `access`). The list needs a three-table JOIN and the viewer's permissions, so it gets its own repository and service; no existing constructor changed. The repository selects only the columns the list renders, so the description, the link and every donor setting never leave the database here. Donation counts for a page come from one grouped query (`donation_repository::count_by_campaign_ids()`), only for campaigns that show them.

Shared parts. The percentage, bar step and "reached" state come from `campaign_service::progress()` (moved out of `viewtopic_listener`; "reached" now needs a target above zero). The bar markup is one partial, `donationcampaigns_progress.html`, included by the box and the list with `{% include ... with {...} only %}`; the tests render it with phpBB's own engine.

Consequences. Off by default, so an update publishes nothing. One more listener on every page (`core.page_header`), idle while the switch is off. The navbar entry is shown whenever the switch is on, even to a viewer for whom the list is empty — checking would cost a query on every page.

ADR-019 — Creating a campaign from the posting form

Decision (1.0.0-beta3). When a manager starts a NEW topic, the posting form offers a "Donation campaign" tab next to "Poll creation". Its panel holds an "attach" checkbox and the same campaign fields as the frontend form. Ticked, the fields are validated with the post, and the campaign is created right after the topic. Decisions D1–D8 (`plans/2026-10-04-posting-form-campaign.md`) plus the owner's answers WD1–WD4:

- **New topics only** (`mode = post`, D1). Editing stays with the management landing; offering the fields when editing the first post would be a second edit path for the same campaign.
- **Who:** `access::can_manage()` in the posting forum (D2) — checked in each of the three events, never inferred from the panel having been shown, because the fields can be posted by hand. Fields injected into a reply, quote or edit, or posted without the permission, are ignored (tests for each).
- **The checkbox is the switch** (D3): unticked, nothing is validated or created.
- **Field names carry the prefix** `donationcampaigns_` (WD4). The posting form is the most shared field namespace among extensions. The fields live in one include, `donationcampaigns_campaign_fields.html`, with a `prefix` parameter: `'` in the frontend form (names unchanged — verified byte-identical on the board) and `'donationcampaigns_'` in the panel. `service/campaign_form.php` reads either.
- **Title:** an empty campaign title becomes the topic subject at submit, decoded with `htmlspecialchars_decode(..., ENT_COMPAT)`, because core stores the subject escaped and the campaign title is stored raw (WD1). There is no prefill: a prefilled value would drift when the subject changes after a preview. The frontend form still requires a title.
- **Validation** on submit and on preview, like core's poll checks, through `campaign_service::validate_fields()` — the field rules without the topic rules, which cannot run before the topic exists. `validate()` was split into three parts composed in the original order; a golden test of 648 input combinations, committed before the split, proves the result unchanged, including the error order that `assert_valid()` depends on.
- **Creation** through `campaign_service::create_campaign()` with the full rules, and the same moderator-log entry as a frontend create (D8). This listener is not a second write path: it reaches the one write path from a second place.
- **Approval queue** (D7): `submit_post()` inserts a queued topic too, and the event runs before core's approval message, so the campaign is created and becomes visible with its topic. Disapproval deletes the topic through `delete_topics()`, and the existing cascade removes the campaign (verified live).

Preview, errors, drafts, double submit. A redisplay shows what was entered (a hidden `donationcampaigns_panel` field marks it). Drafts store only subject and message, so the panel is lost on "Save draft" and empty on "Load draft" (D6). A resubmitted form is stopped by core: the flood check, then `posting.lock`, whose lifetime is `flood_interval`. On a board

with `flood_interval = 0` core itself posts the topic twice; each topic then gets its own campaign — no conflict, no error (observed live).

Failure after the post is saved (WD2). Nothing is rolled back. If `create_campaign()` throws, a `critical` entry (`LOG_DONATIONCAMPAIGNS_POSTING_CREATE_FAILED`) is written and, for an approved topic, the event variable `redirect_url` is pointed at the management landing, whose form offers a second try; for a queued topic core shows its fixed message after the event, so only the log entry. **Limit:** a database error in phpBB is not an exception — the DBAL calls `trigger_error(E_USER_ERROR)`, core shows its "General Error" page and logs `LOG_GENERAL_ERROR` itself. The topic is saved without a campaign in that case too, and the landing still offers to create it; intercepting it would mean bypassing the DBAL.

Shared parts. `campaign_form` (request reading, target parsing, the "amount error replaces TARGET_POSITIVE" merge, new-campaign defaults) moved unchanged out of `campaign_controller`; the controller tests are unchanged.

ADR-020 — Templates in native Twig only; template tests render with phpBB's engine

Decision (1.0.0-beta4). Every template is written in native Twig. The spelling is exactly what phpBB's lexer produces from the legacy syntax — `{{ VAR|e }}`, `{{ lang('KEY') }}``{{ lang('COLON') }}`, `{% if %}`, `{% for row in loops.row %}`, `loops.row|length` — with one deliberate exception: the core header, footer and pagination are included with Twig's lower-case `{% include '...' %}`, as core's Twig templates and the Skeleton Extension do. `{% INCLUDECSS %}` and `{% INCLUDEJS %}` stay upper case: they are phpBB's own tags and have no Twig counterpart. Developer notes are Twig comments `{# ... #}`; nothing in a shipped template starts with `<!--`.

Why. The phpBB Extension Check team pointed at the legacy syntax, phpBB's documentation ("Tutorial: Template syntax") says it will be deprecated and recommends Twig, and the official Skeleton Extension writes Twig only. prosilver is **not** the reference for template syntax: in 3.3 nearly all of its templates still use the legacy syntax. phpBB keeps accepting the legacy syntax by rewriting it in `phpbb/template/twig/lexer.php` on every compile — including inside HTML comments, which is how a comment that named core's `{MESSAGE}` printed the post text into the posting form's page source (F1, fixed in beta4). The lexer still runs over native Twig, so a `{WORD}` anywhere in a template, a comment included, is still rewritten; the guard forbids it.

How it was shown to change nothing. Each conversion commit was checked by compiling old and new templates with phpBB's lexer and Twig 2.16.1 and comparing the PHP (`tools/beta4/compare-compiled.php`; identical except the generated class name and the inert namespace wrapper of `INCLUDE`), and by capturing 46 page states on the Docker board before and after (`tools/beta4/render-snapshot.sh`, data from `tools/beta4/seed-board.sh`); every capture was byte-identical.

Tests. Template tests render with phpBB's real engine (`tests/template_renderer.php`), built as phpBB builds it for its own template tests (`tests/template/template_test_case.php`): environment, twig, extension, lexer and context from the read-only phpBB test tree, on that tree's Twig. No copy of phpBB code, no Twig in require-dev; `tests/bootstrap.php` stops the suite on any phpBB other than 3.3.17 or Twig other than 2.16.1, it never skips. Consequences: conditions are evaluated, block rows carry phpBB's row variables, the extension's language keys render as keys (core's own `common.php` is loaded by phpBB on first use, so tests assert only the extension's keys or structure, never core texts), and the core header/footer are stubs carrying only `{ $STYLESHEETS } / { $SCRIPTS }`, so `INCLUDECSS/INCLUDEJS` output is visible. Rules about the source read the template as the lexer hands it to Twig (`template_renderer::lexed()`), so they hold for either spelling.

Guards. `architecture_test::test_templates_use_native_twig_only` rejects every legacy construct the lexer would rewrite, comments included; `test_templates_carry_no_html_comments` rejects any `<!--`; the HTML5 rule rejects a space before the end of a tag.

Styles

prosilver only, for version 1.0 (ADR-013). Templates under `styles/prosilver/template/`, in native Twig (ADR-020); stylesheet under `styles/prosilver/theme/`, included with `{% INCLUDECSS %}` from a header template event.

No inline CSS, no inline JavaScript, and **nothing requires JavaScript** — the progress bar's width is a CSS class per five-percent step rather than an inline style. All custom classes carry the `donationcampaigns-` prefix. No fixed widths, no absolute positioning.

Tests

```
php vendor/bin/phpunit          # everything
php vendor/bin/phpunit --testsuite unit      # pure logic + architectural guards
php vendor/bin/phpunit --testsuite migration # schema, config, permission, modules
php vendor/bin/phpunit --testsuite repository
php vendor/bin/phpunit --testsuite service   # rules, transactions, total integrity
php vendor/bin/phpunit --testsuite event     # listeners + shipped prosilver assets
php vendor/bin/phpunit --testsuite acp       # ACP modules + adm templates
php vendor/bin/phpcs --standard=phpcs.xml
find ext -name '*.php' -print0 | xargs -0 -n1 php -l
```

`phpbb_database_test_case` is unusable — it extends `PHPUnit\DbUnit\TestCase`, and dbunit is abandoned and incompatible with PHPUnit 9. Database tests therefore build their schema from phpBB's own baseline migration (`\phpbb\db\migration\data\v30x\release_3_0_0`) and drive phpBB's real tools. The 3.0.0 baseline predates the 3.1 visibility columns, so fixtures that drive core's deletion paths add them explicitly.

Template tests render with phpBB's real engine (`tests/template_renderer.php`, ADR-020): `render()` for what a page shows, `lexed()` for rules about the source, `inline_partials()` where a test reads a form's source with its shared fields. The phpBB test tree must be 3.3.17 with its `vendor/` installed (CI: `composer install --no-dev` there); the bootstrap stops otherwise.

Integration tests call **core's own functions** — `delete_topics()`, `delete_posts()`, `prune()`, `auto_prune()`, `acp_forums::delete_forum()` — through a real dispatcher with the listeners subscribed, rather than invoking the listeners directly. The assumption worth testing is not that our code works but that core reaches it.

Database coverage

Engine	Status
SQLite 3	Every automated test
MariaDB 10.11	Live integration: topic deletion, forum deletion with and without attachments, donations-first ordering observed in the query log, no orphans, migration idempotence — and the <code>mysqli</code> branch of forum deletion that SQLite never executes
MySQL, PostgreSQL, MS SQL, Oracle	Not executed. DBAL-only SQL, so expected to work; not verified

Local integration environment

A disposable Docker board (phpBB 3.3.17, PHP 8.2, MariaDB 10.11) is used to verify UI work against a real installation, since automated tests cannot see a template that fails to compile or a page that renders wrongly.

It lives outside the extension package and is not part of the repository. Its setup notes live with it. **No credentials, hostnames or local paths belong in this repository.**

EPV and coding standards

```
docker compose exec web /opt/epv/vendor/bin/EPV.php run --dir=/var/www/html/phpBB/ext/
```

EPV must be pointed at the directory *containing* `vendor/package/`, not at the extension root, or it reports a spurious packaging error. The current state is **0 fatal, 0 errors, 0 warnings, 0 notices**.

Two of its rules shaped the code:

- `htmlspecialchars()` is an **Error** in EPV's eyes, whatever the context. Hence `|e` in templates and `utf8 htmlspecialchars()` at sinks.
- Its **SQL-injection check is a regex** requiring `' . $`, skipped when the line contains `sql_in_set`, `sql_escape` or several other DBAL helpers. An inline `(int)` cast breaks the pattern — which is why queries are written `= ' . (int) $id` rather than casting on an earlier line.

EPV passing is not proof of submission-policy compliance. It is a mechanical scanner. The phpBB validation and development policies have not been reviewed for this extension, and no claim about them is made.